OPERATING SYSTEMS · PRODUCT NO. 10

LabOS

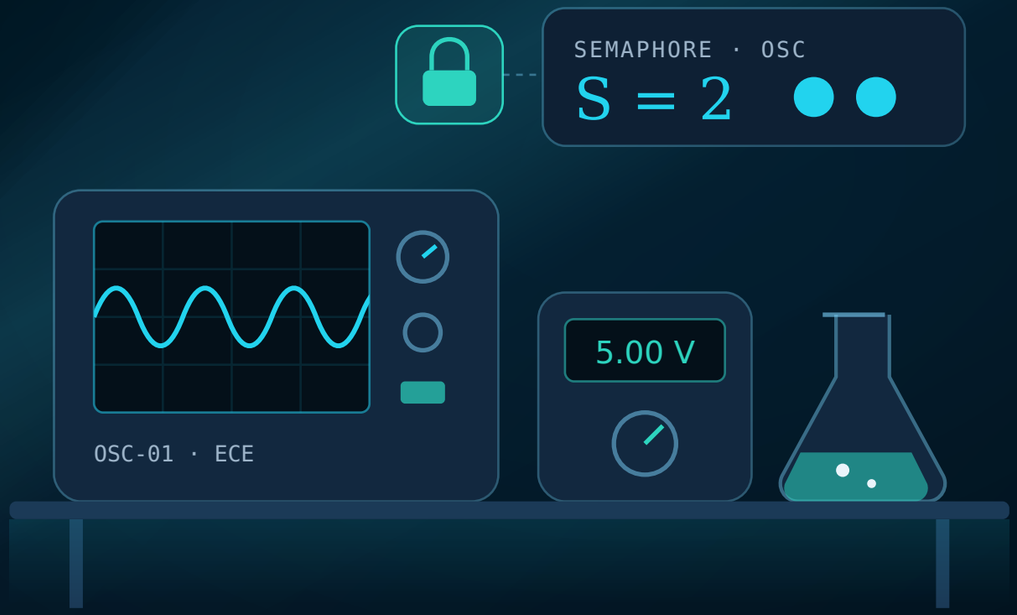
Fair, live and conflict-free sharing of engineering lab equipment.

An **Engineering Laboratory Equipment Allocation System**: a Flask web application that books lab equipment using the same tools an operating system uses to share resources — mutex locks, counting semaphores and FIFO waiting queues.

Python · Flask · threading · HTML · CSS · JS · VS Code

PRESENTED BY

G. Vishwesh	2503A52889
P. Anil Teja	2503A52890
T. Chandrika	2503A52887
M. Laxmi Prasanna Reddy	2503A52885
P. Somnath Reddy	2503A52888



6	53	156
DEPARTMENTS	EQUIPMENT TYPES	TOTAL UNITS

PRODUCT NO. 10 · PROJECT TEAM

Meet the people behind *LabOS*

Five members, each owning a key part of the Engineering Laboratory Equipment Allocation System.

01 / LEAD

GV

G. Vishwesh

2503A52889

Team Lead & System Architect

02 / BACKEND

AT

P. Anil Teja

2503A52890

Backend Developer

03 / OS

TC

T. Chandrika

2503A52887

OS Synchronization Developer

04 / FRONTEND

LP

M. Laxmi Prasanna Reddy

2503A52885

Frontend Developer

05 / QA

SR

P. Somnath Reddy

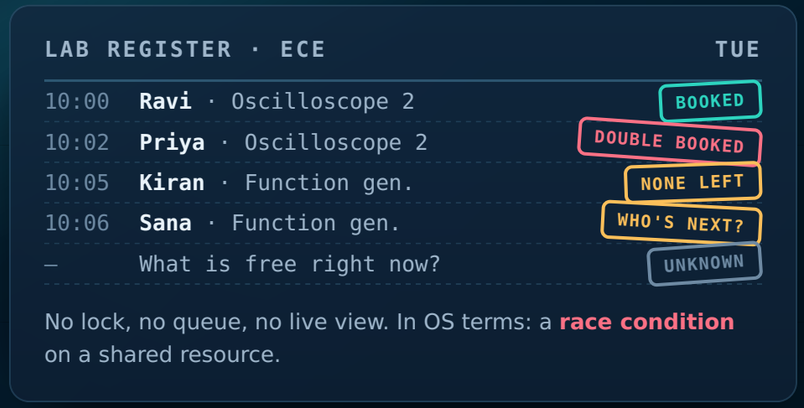
2503A52888

Testing & Documentation Lead

01 PROBLEM STATEMENT

Many students, a few instruments, and *one paper register*

- **Double bookings.** Two students can write their names against the same unit, and nobody notices until both arrive.
- **Limited equipment.** A lab may have only two oscilloscopes for a whole batch, so demand always exceeds supply.
- **Unfair waiting.** When a unit is free, whoever is standing closest gets it. There is no recorded order.
- **No live availability.** Students must walk to the lab or ask the assistant just to find out what is free.



02 PROPOSED SOLUTION

Treat lab equipment like an OS treats a shared resource: *synchronize every request*

Every booking goes through one Flask backend, where a lock, a semaphore and a queue decide the outcome in a fixed, fair order.

fixes: paper register · no live view

Automated booking & live monitoring

Students book from a web form. The dashboard shows available units, holders and the queue as they change.

`book_equipment()`

fixes: double bookings

Mutex lock

Only one request at a time may check and update availability, so two students can never receive the same unit.

`threading.Lock()`

fixes: over-allocation

Counting semaphores

One semaphore per equipment type, started at its unit count. It can never hand out more units than exist.

`BoundedSemaphore(n)`

fixes: unfair waiting

FIFO waiting queue

When no unit is free, students wait in arrival order. A returned unit goes straight to the front of the queue.

`collections.deque`

IMPLEMENTED Booking, return, per-type semaphores, mutex-protected critical section, FIFO queues, automatic allocation on return, live dashboard and allocation history.

FUTURE SCOPE Database persistence, login, notifications and booking time limits. These are not part of the current version.

03 MAIN FEATURES

Eight features, from the *booking form* to the *history log*

01

Student booking

Student enters their details and requests a piece of equipment.

`POST → book_equipment()`

02

Department selection

Equipment is grouped under six departments, so the list stays short.

`6 departments`

03

Equipment availability

Free units out of total, shown per equipment type.

`available / total`

04

Waiting queue

If no unit is free, the student gets a position in a FIFO queue.

`deque.append()`

05

Automatic allocation

A returned unit is handed to the first waiting student with no manual step.

`deque.popleft()`

06

Return functionality

Returning frees the unit or passes it to the queue, inside the lock.

`return_equipment()`

07

Live monitoring

Dashboard reflects holders, counts and queues without reloading the page.

`JavaScript fetch → Flask`

08

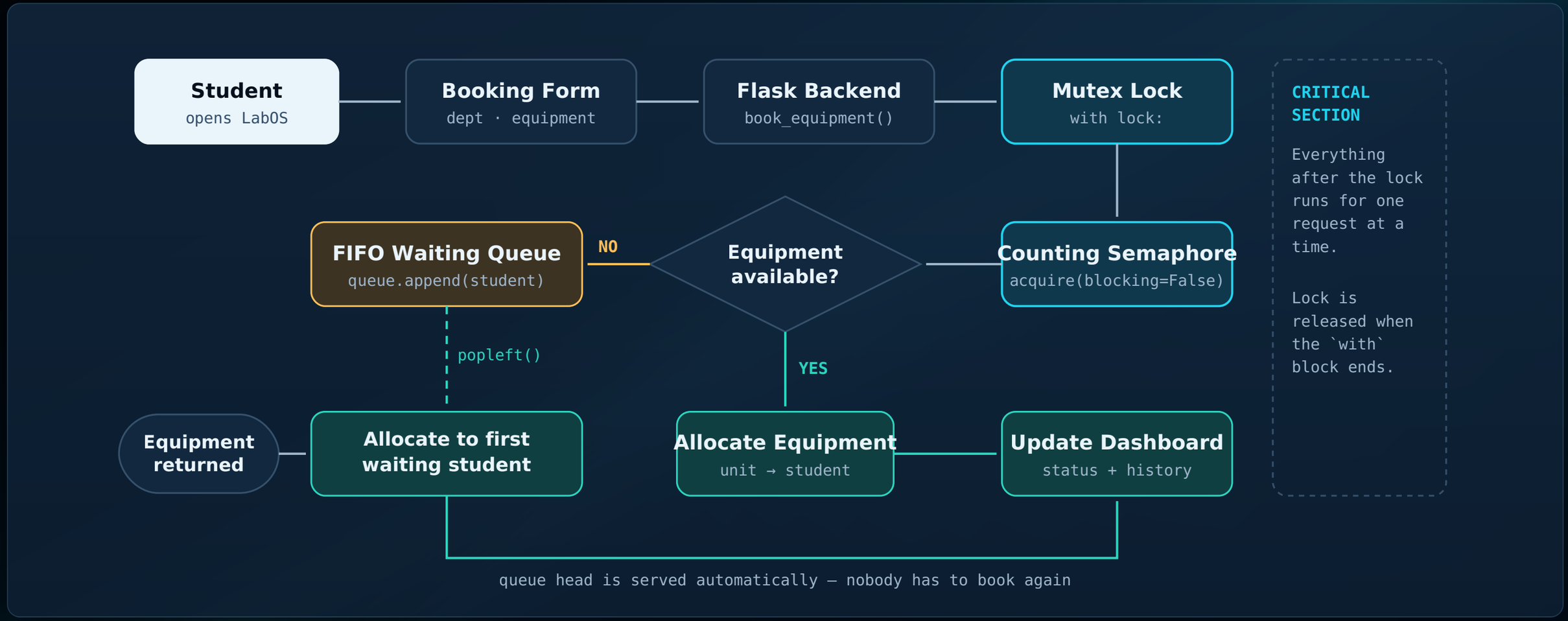
Allocation history

Every allocation and return is recorded for the lab in-charge to review.

`history log`

04 SYSTEM WORKFLOW

One request, from the *booking form* to the *dashboard*



Synchronization step Allocation Waiting

05 OPERATING SYSTEMS CONCEPTS

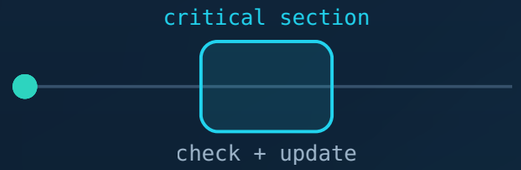
Six OS ideas, each doing *one clear job* in LabOS



Mutex Lock

`threading.Lock()`

Only one thread owns the lock. Others wait until it is released, so booking data is never changed twice at once.



Critical Section

`check → allocate`

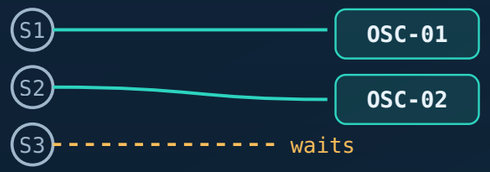
The code that reads the free count and assigns a unit. It must run for one request at a time, or two could take the last unit.



Counting Semaphore

`BoundedSemaphore(n)`

A counter of free units. Acquire takes one, release gives one back. At zero, no more units can be given out.



Resource Allocation

`units → students`

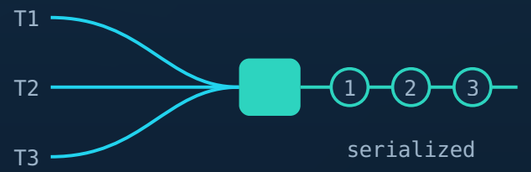
Each unit is assigned to exactly one student. When units run out, the request is not rejected; it waits.



FIFO Queue

`collections.deque`

First in, first out. Students join at the rear and are served from the front, so waiting order is always fair.



Concurrency Control

`Flask threads`

Flask can serve requests on several threads. The lock turns simultaneous requests into a safe, one-by-one order.

06 LIVE DEMONSTRATION

Two oscilloscopes, many students: *try it yourself*

Oscilloscope · ECE

BoundedSemaphore(2)

S = 2

Mutex free

UNITS · CLICK A BUSY UNIT TO RETURN IT

WAITING FOR THE MUTEX

Rules: **acquire()** succeeds only while S > 0. On return, the unit goes to the **front of the queue** first; S rises only when nobody is waiting.

FIFO waiting queue

0 waiting

Book Equipment

Return Equipment

Reset Simulation

Rush ×3

07 PYTHON IMPLEMENTATION & ARCHITECTURE

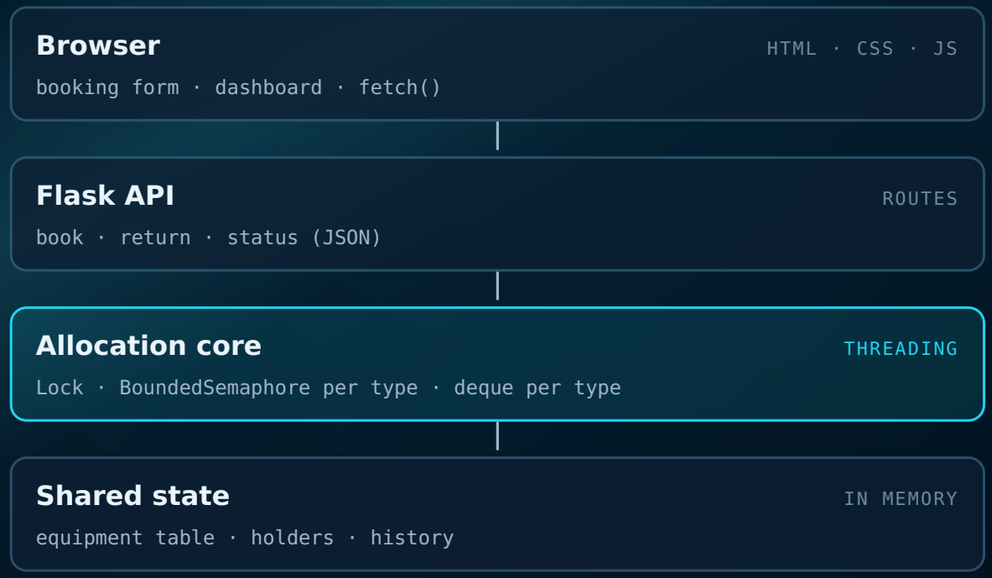
Two functions, one lock, *one semaphore per equipment type*

app.py · simplified excerpt

```
lock = threading.Lock()            # one mutex for shared state
equipment = {
    "Oscilloscope": {"sem": threading.BoundedSemaphore(2),
                     "queue": deque(), "holders": []},
    # ... 53 equipment types in total
}

def book_equipment(student, item):
    with lock:                     # enter critical section
        eq = equipment[item]
        if eq["sem"].acquire(blocking=False):
            eq["holders"].append(student)
            return "ALLOCATED"
        eq["queue"].append(student)        # FIFO: join the rear
        return f"WAITING #{len(eq['queue'])}"

def return_equipment(student, item):
    with lock:
        eq = equipment[item]
        eq["holders"].remove(student)
        if eq["queue"]:                    # someone is waiting
            eq["holders"].append(eq["queue"].popleft())
        else:
            eq["sem"].release()            # unit back in the pool
```



Flask endpoint — excerpt

```
@app.route("/book", methods=["POST"])
def book():
    d = request.get_json()
    result = book_equipment(d["student"], d["equipment"])
    return jsonify(status=result)
```

08 DASHBOARD & APPLICATION

What students and the lab in-charge *actually see*

LIVE DASHBOARD

Dashboard screenshot

Availability, holders and waiting queues

click to insert an image

6 DEPARTMENTS	53 EQUIPMENT TYPES	156 TOTAL UNITS
ECE Electronics & Comm.	**EEE** Electrical & Electronics	**MECH** Mechanical
CIVIL Civil	**CSE/IT** Computer Science & IT	**SCI** Science labs

STUDENT INPUTS

Student name · Enrollment no. · Department · Equipment · Book / Return

BOOKING FORM

Booking form screenshot

click to insert an image

09 TESTING & TEST CASES

Six scenarios that check the *synchronization rules*, not just the screens

ID	TEST CASE	SCENARIO	EXPECTED BEHAVIOUR	RESULT
TC-01	**Successful allocation**	Book an Oscilloscope while 2 of 2 are free	Unit allocated; semaphore 2 → 1; dashboard shows the holder	✓ Pass
TC-02	**Capacity enforcement**	Third booking when both units are taken	No third unit is given; semaphore never goes below 0	✓ Pass
TC-03	**FIFO waiting queue**	Kiran, then Sana, book while count is 0	Queue order is Kiran (#1), Sana (#2)	✓ Pass
TC-04	**Allocation after return**	A holder returns with Kiran at the front	Unit goes to Kiran automatically; Sana moves to #1	✓ Pass
TC-05	**Independent semaphores**	Oscilloscopes at 0, then book a different equipment type	Booking succeeds; other types are not affected	✓ Pass
TC-06	**Concurrent requests**	Several booking requests sent at the same moment	No unit is given to two students; allocations never exceed capacity	✓ Pass

Each case was run in the LabOS application and checked against the dashboard and allocation history.

10 CHALLENGES, LIMITATIONS & FUTURE SCOPE

What was hard, what we built, and *what comes next*

Concurrent access

Two requests could both see “1 free” and both take it.

Handled: check and allocate run inside one mutex-protected critical section.

Waiting queue data

Keeping the order correct when students join and leave.

Handled: one deque per equipment type; append() to join, popleft() to serve.

Equipment identification

156 units across 53 types: returns must free the right one.

Handled: each allocation is recorded against its equipment type and student.

Dynamic dashboard

Showing changes without the user reloading the page.

Handled: JavaScript fetches the latest status from Flask and redraws.

Implemented today

working

- ✓ **Booking & return** through the Flask backend
- ✓ **Mutex + per-type counting semaphores**
- ✓ **FIFO queues** with automatic allocation on return
- ✓ **Live dashboard** and allocation history

Future scope

not built yet

- → **Database persistence**: today, data lives in memory and resets on restart
- → **Authentication** for students and lab in-charges
- → **Notifications** when a queued student receives a unit
- → **Booking time limits** so units are returned on schedule

11 CONCLUSION

OS synchronization, applied to *a real lab problem*

- **Mutex lock** removes double bookings by guarding the critical section.
- **Counting semaphores** make sure no more units are given out than exist.
- **FIFO queues** make waiting fair and allocation automatic.
- **A live dashboard** replaces the paper register and the walk to the lab.



Thank *you*

Questions are welcome.

LabOS · Product No. 10 · Operating Systems

G. Vishwesh · P. Anil Teja · T. Chandrika · M. Laxmi Prasanna Reddy · P. Somnath Reddy